UNIT IV

Associative Memory

An associative memory network can store a set of patterns as memories. When the associative memory is being presented with a key pattern, it responds by producing one of the stored patterns, which closely resembles or relates to the key pattern. Thus, the recall is through association of the key pattern, with the help of information memorized. These types of memories are also called as *content-addressable memories* (CAM) in contrast to that of traditional *address-addressable memories* in digital computers where stored pattern (in bytes) is recalled by its address. It is also a matrix memory as in RAM/ROM. The CAM can also be viewed as associating data to address, i.e., for every data in the memory there is a corresponding unique address. Also, it can be viewed as data correlator. Here input data is correlated with that of the stored data in the CAM. It should be noted that the stored patterns must be unique, i.e., different patterns in each location. If the same pattern exists in more than one location in the CAM, then, even though the correlation is correct, the address is noted to be ambiguous. The basic structure of CAM is given in Figure 4-1.

Associative memory makes a parallel search within a stored data file. The concept behind this search is to output any one or all stored items which match the given search argument and to retrieve the stored data either completely or partially.

Two types of associative memories can be differentiated. They are *autoassociative memory* and *heteroassociative memory*. Both these nets are single-layer nets in which the weights are determined in a manner that the net stores a set of pattern associations. Each of this association is an input–output vector pair, say, s:t. If each of the output vectors is same as the input vectors with which it is associated, then the net is a said to

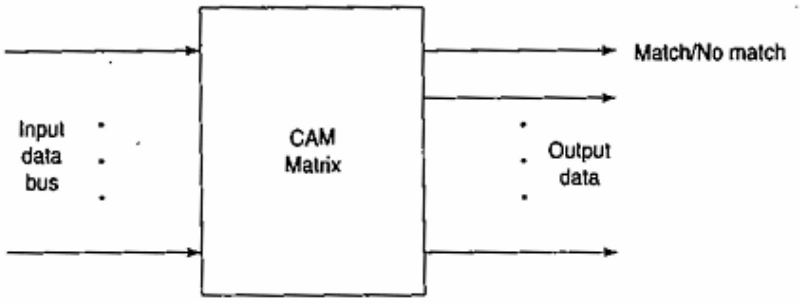


Figure 4-1 CAM architecture.

be autoassociative memory net. On the other hand, if the output vectors are different from the input vectors then the net is said to be heteroassociative memory net.

If there exist vectors, say, $x = (x_1, x_2, \ldots, x_n)^T$ and $x' = (x_1', x_2', \ldots, x_n')^T$, then the hamming distance (HD) is defined as the number of mismatched components of x and x' vectors, i.e.,

$$\text{HD}(x, x') = \begin{cases} \sum_{i=1}^{n} |x_i - x_i'| & \text{if } x_i, x_i' \in [0, 1] \\ \frac{1}{2} \sum_{i=1}^{n} |x_i - x_i'| & \text{if } x_i, x_i' \in [-1, 1] \end{cases}$$

The architecture of an associative net may be either feed-forward or iterative (recurrent). As is already known, in a feed-forward net the information flows from the input units to the output units; on the other hand, in a recurrent neural net, there are connections among the units to form a closed-loop structure. In the forthcoming sections, we will discuss the training algorithms used for pattern association and various types of association nets in detail.

4.2 Training Algorithms for Pattern Association

There are two algorithms developed for training of pattern association nets. These are discussed below.

4.2.1 Hebb Rule

The Hebb rule is widely used for finding the weights of an associative memory neural net. The training vector pairs here are denoted as *s:t*. The flowchart for the training algorithm of pattern association is as shown in Figure 4-2. The weights are updated until there is no weight change. The algorithmic steps followed are given below:

Step 0: Set all the initial weights to zero, i.e.,

$$w_{ij} = 0 \quad (i = 1 \text{ to } n, j = 1 \text{ to } m)$$

Step 1: For each training target input output vector pairs *s:t*, perform Steps 2–4.

Step 2: Activate the input layer units to current training input,

$$x_i = s_i \quad (\text{for } i = 1 \text{ to } n)$$

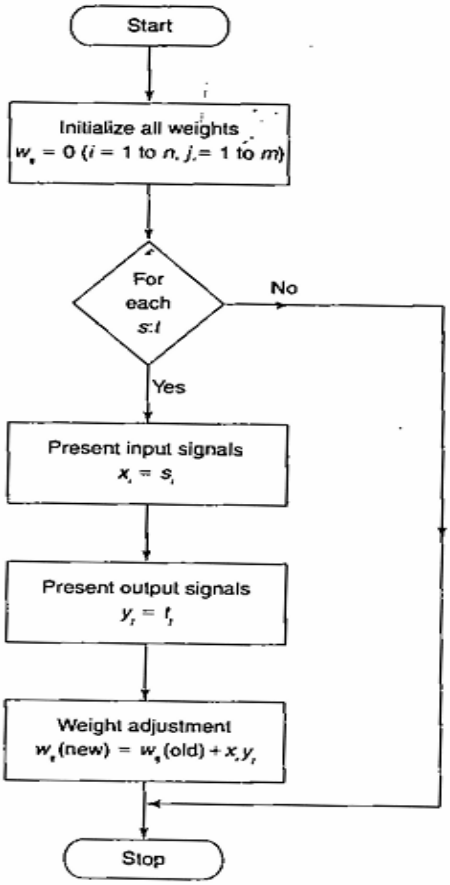


Figure 4-2 Flowchart for Hebb rule.

Step 3: Activate the output layer units to current target output,

$$y_j = t_j \quad (\text{for } j = 1 \text{ to } m)$$

Step 4: Start the weight adjustment

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + x_i y_j \quad (\text{for } i = 1 \text{ to } n, j = 1 \text{ to } m)$$

This algorithm is used for the calculation of the weights of the associative nets. Also, it can be used with patterns that are being represented as either binary or bipolar vectors.

4.2.2 Outer Products Rule

Outer products rule is an alternative method for finding weights of an associative net. This is depicted as follows:

$$\text{Input} \Rightarrow s = (s_1, \ldots, s_i, \ldots, s_n)$$
$$\text{Output} \Rightarrow t = (t_1, \ldots, t_j, \ldots, t_m)$$

The outer product of the two vectors is the product of the matrices $S = s^T$ and $T = t$, i.e., between $[n \times 1]$ matrix and $[1 \times m]$ matrix. The transpose is to be taken for the input matrix given.

The matrix multiplication is done as follows:

$$ST = s^T t$$

$$= \begin{bmatrix} s_1 \\ \cdot \\ \cdot \\ \cdot \\ s_i \\ \cdot \\ \cdot \\ \cdot \\ s_n \end{bmatrix}_{n \times 1} [t_1 \ldots t_j \ldots t_m]_{1 \times m}$$

$$W = \begin{bmatrix} s_1 t_1 & \ldots & s_1 t_j & \ldots & s_1 t_m \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ s_i t_1 & \ldots & s_i t_j & \ldots & s_i t_m \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ s_n t_1 & \ldots & s_n t_j & \ldots & s_n t_m \end{bmatrix}_{n \times m}$$

This weight matrix is same as the weight matrix obtained by Hebb rule to store the pattern association $s:t$. For storing a set of associations, $s(p):t(p)$, $p = 1$ to P, wherein,

$$s(p) = (s_1(p), \ldots, s_i(p), \ldots, s_n(p))$$
$$t(p) = (t_1(p), \ldots, t_j(p), \ldots, t_m(p))$$

the weight matrix $W = \{w_{ij}\}$ can be given as

$$w_{ij} = \sum_{p=1}^{P} s_i^T(p)\, t_j(p)$$

This can also be rewritten as

$$W = \sum_{p=1}^{P} s^T(p)\, t(p)$$

---

for finding the weights of the net using Hebbian learning. Similar to the Hebb rule, even the delta rule discussed in Chapter 2 can be used for storing weights of pattern association nets.

4.3 Autoassociative Memory Network

4.3.1 Theory

In the case of an autoassociative neural net, the training input and the target output vectors are the same. The determination of weights of the association net is called storing of vectors. This type of memory net needs suppression of the output noise at the memory output. The vectors that have been stored can be retrieved from distorted (noisy) input if the input is sufficiently similar to it. The net's performance is based on its ability to reproduce a stored pattern from a noisy input. It should be noted, that in the case of autoassociative net, the weights on the diagonal can be set to zero. This can be called as auto associative net with no self-connection. The main reason behind setting the weights to zero is that it improves the net's ability to generalize or increase the biological plausibility of the net. This may be more suited for iterative nets and when delta rule is being used.

4.3.2 Architecture

The architecture of an autoassociative neural net is shown in Figure 4-3. It shows that for an autoassociative net, the training input and target output vectors are the same. The input layer consists of n input units and the output layer also consists of n output units. The input and output layers are connected through weighted interconnections. The input and output vectors are perfectly correlated with each other component by component.

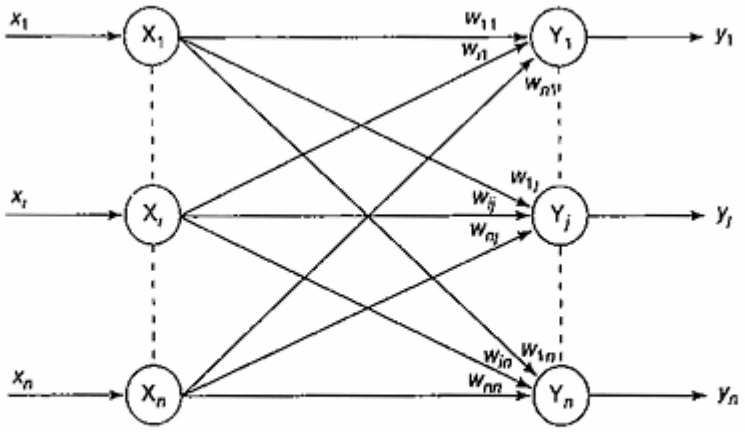


Figure 4-3 Architecture of autoassociative net.

Flowchart for training process of Autoassociative Memory Network

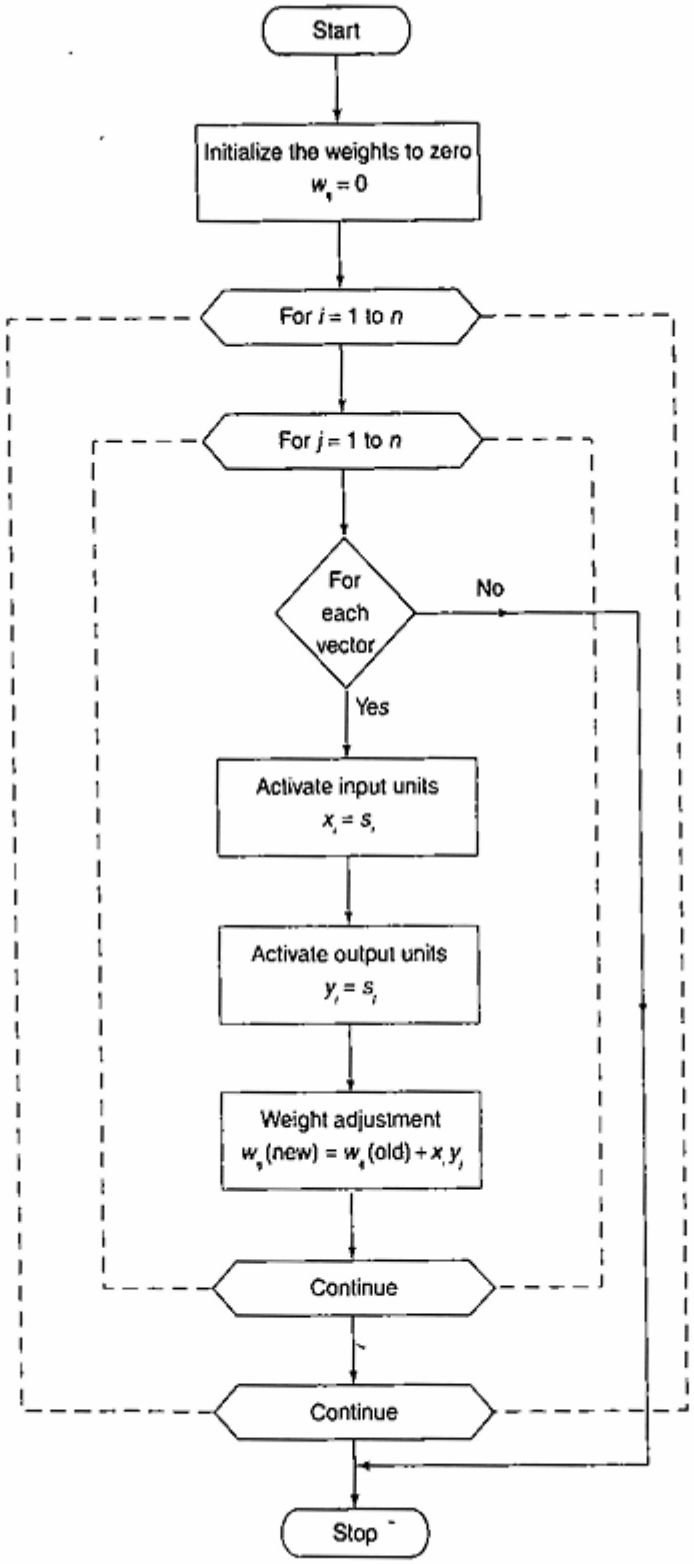


Figure 4-4 Flowchart for training of autoassociative net.

4.3.4 Training Algorithm

The training algorithm discussed here is similar to that discussed in Section 4.2.1 but there are same numbers of output units as that of the input units.

Step 0: Initialize all the weights to zero,

$$w_{ij} = 0 \ (i = 1 \text{ to } n, \ j = 1 \text{ to } n)$$

Step 1: For each of the vector that has to be stored perform Steps 2–4.

Step 2: Activate each of the input unit,

$$x_i = s_i \ (i = 1 \text{ to } n)$$

Step 3: Activate each of the output unit,

$$y_j = s_j \ (j = 1 \text{ to } n)$$

Step 4: Adjust the weights,

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + x_i y_j$$

The weights can also be determined by the formula

$$W = \sum_{p=1}^{P} s^T(p) s(p)$$

4.3.5 Testing Algorithm

An autoassociative memory neural network can be used to determine whether the given input vector is a "known" vector or an "unknown" vector. The net is said to recognize a "known" vector if the net produces a pattern of activation on the output units which is same as one of the vectors stored in it. The testing procedure of an autoassociative neural net is as follows:

Step 0: Set the weights obtained for Hebb's rule or outer products.

Step 1: For each of the testing input vector presented perform Steps 2–4.

Step 2: Set the activations of the input units equal to that of input vector.

Step 3: Calculate the net input to each output unit $j = 1$ to n:

$$y_{inj} = \sum_{i=1}^{n} x_i w_{ij}$$

Step 4: Calculate the output by applying the activation over the net input:

$$y_j = f(y_{inj}) = \begin{cases} +1 & \text{if } y_{inj} > 0 \\ -1 & \text{if } y_{inj} \leq 0 \end{cases}$$

This type of network can be used in speech processing, image processing, pattern classification, etc.

4.4 Heteroassociative Memory Network

4.4.1 Theory

In case of a heteroassociative neural net, the training input and the target output vectors are different. The weights are determined in a way that the net can store a set of pattern associations. The association here is a pair of training input target output vector pairs $(s(p), t(p))$, with $p = 1, \ldots, P$. Each vector $s(p)$ has n components and each vector $t(p)$ has m components. The determination of weights is done either by using Hebb rule or delta rule. The net finds an appropriate output vector, which corresponds to an input vector x, that may be either one of the stored patterns or a new pattern.

4.4.2 Architecture

The architecture of a heteroassociative net is shown in Figure 4-5. From the figure, it can be noticed that for a heteroassociative net, the training input and target output vectors are different. The input layer consists of n number of input units and the output layer consists of m number of output units. There exist weighted interconnections between the input and output layers. The input and output layer units are not correlated with each other. The flowchart of the training process and the training algorithm are the same as discussed in Section 4.2.1.

4.4.3 Testing Algorithm

The testing algorithm used for testing the heteroassociative net with either noisy input or with known input is as follows:

Step 0: Initialize the weights from the training algorithm.

Step 1: Perform Steps 2–4 for each input vector presented.

Step 2: Set the activation for input layer units equal to that of the current input vector given, x_i.

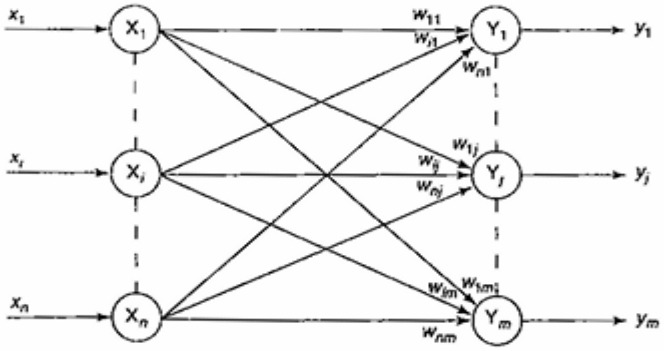


Figure 4-5 Architecture of heteroassociative net.

Step 3: Calculate the net input to the output units:

$$y_{inj} = \sum_{i=1}^{n} x_i w_{ij} \quad (j = 1 \text{ to } m)$$

Step 4: Determine the activations of the output units over the calculated net input:

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > 0 \\ 0 & \text{if } y_{inj} = 0 \\ -1 & \text{if } y_{inj} < 0 \end{cases}$$

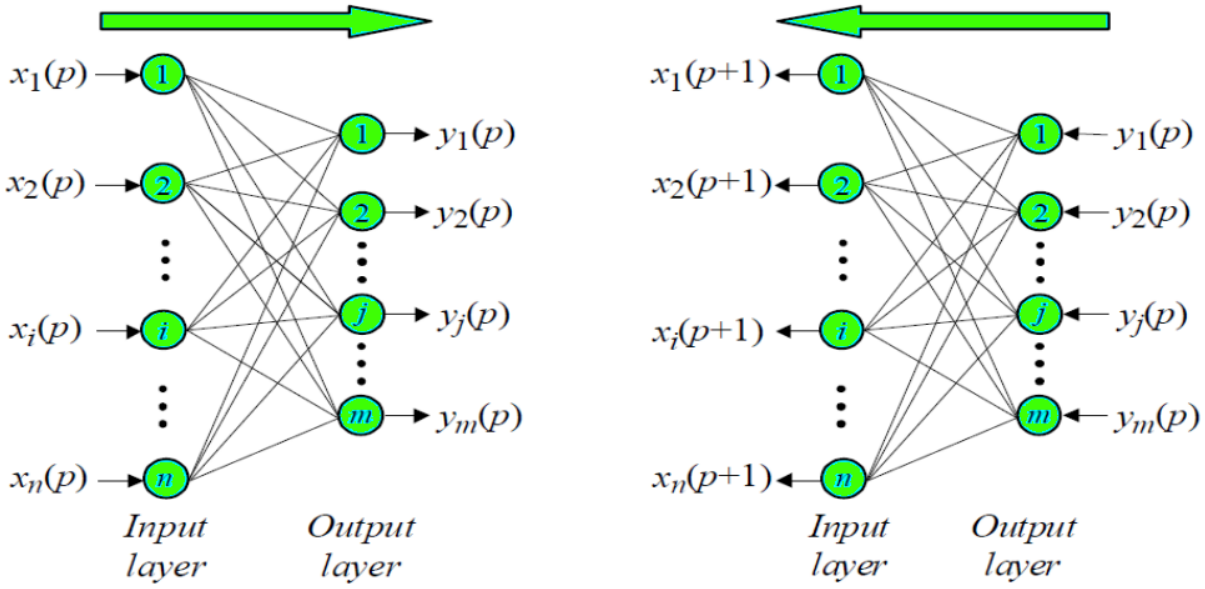
Thus, the output vector y obtained gives the pattern associated with the input vector x.

Note: Heteroassociative memory is not an iterative memory network. If the responses of the net are binary, then the activation function to be used is

$$y_j = \begin{cases} 1 & \text{if } y_{inj} \geq 0 \\ 0 & \text{if } y_{inj} < 0 \end{cases}$$

Bidirectional Associative Memory

Bidirectional Associative Memory (BAM) is a supervised learning model in Artificial Neural Network. This is *hetero-associative memory*, for an input pattern, it returns another pattern which is potentially of a different size. This phenomenon is very similar to the human brain. Human memory is necessarily associative. It uses a chain of mental associations to recover a lost memory like associations of faces with names, in exam questions with answers, etc. In such memory associations for one type of object with another, a Recurrent Neural Network (RNN) is needed to receive a pattern of one set of neurons as an input and generate a related, but different, output pattern of another set of neurons. Why BAM is required? The main objective to introduce such a network model is to store hetero-associative pattern pairs. This is used to retrieve a pattern given a noisy or incomplete pattern. BAM Architecture: When BAM accepts an input of *n*-dimensional vector *X* from set *A* then the model recalls *m*-dimensional vector *Y* from set *B*. Similarly when *Y* is treated as input, the BAM recalls *X*.



(*a*) Forward direction. (*b*) Backward direction.

Algorithm:

1. Storage (Learning): In this learning step of BAM, weight matrix is calculated between M pairs of patterns (fundamental memories) are stored in the synaptic weights of the network following the equation [Tex]$W=\sum_{m=1}^{M} X_{m} Y_{m}^{T}$[/Tex]
2. Testing: We have to check that the BAM recalls perfectly [Tex]Y_{m}[/Tex] for corresponding [Tex]X_{m}[/Tex] and recalls [Tex]X_{m}[/Tex] for corresponding [Tex]Y_{m}[/Tex].

Using, [Tex]\[Y_{m}=\operatorname{sign}\left(W^{T} X_{m}\right), \quad m=1.2, \ldots, M\]\[X_{m}=\operatorname{sign}\left(W Y_{m}\right), \quad m=1.2, \ldots, M\][/Tex] All pairs should be recalled accordingly.

3. Retrieval: For an unknown vector *X* (a corrupted or incomplete version of a pattern from set *A* or *B*) to the BAM and retrieve a previously stored association: $X \neq X_m, m = 1,2, \ldots, M$

    - *Initialize the BAM:* [Tex] \[X(0)=X, \quad p=0\] [/Tex]
    - *Calculate the BAM output at iteration [Tex]p[/Tex]:* [Tex] \[Y(p)=\operatorname{sign}\left[W^{T} X(p)\right]\] [/Tex]
    - *Update the input vector [Tex]$X(p)$[/Tex]:* [Tex] \[X(p+1)=\operatorname{sign}[W Y(p)]\] [/Tex]
    - *Repeat* the iteration until convergence, when input and output remain unchanged.

Limitations of BAM:

- Storage capacity of the BAM: In the BAM, stored number of associations should not be exceeded the number of neurons in the smaller layer.
- Incorrect convergence: Always the closest association may not be produced by BAM.

4.5 Bidirectional Associative Memory (BAM)

4.5.1 Theory

The BAM was developed by Kosko in the year 1988. The BAM network performs forward and backward associative searches for stored stimulus responses. The BAM is a recurrent heteroassociative pattern-matching network that encodes binary or bipolar patterns using Hebbian learning rule. It associates patterns, say from set A to patterns from set B and vice versa is also performed. BAM neural nets can respond to input from either layers (input layer and output layer). There exist two types of BAM, called *discrete* and *continuous* BAM. These two types of BAM are discussed in the following sections.

4.5.2 Architecture

The architecture of BAM network is shown in Figure 4-6. It consists of two layers of neurons which are connected by directed weighted path interconnections. The network dynamics involve two layers of interaction. The BAM network iterates by sending the signals back and forth between the two layers until all the neurons reach equilibrium. The weights associated with the network are bidirectional. Thus, BAM can respond to the inputs in either layer. Figure 4-6 shows a single layer BAM network consisting of n units in X layer and m units in Y layer. The layers can be connected in both directions (bidirectional) with the result the weight matrix sent from the X layer to the Y layer is W and the weight matrix for signals sent from the Y layer to the X layer is W^T. Thus, the weight matrix is calculated in both directions.

4.5.3 Discrete Bidirectional Associative Memory

The structure of discrete BAM is same as shown in Figure 4-6. When the memory neurons are being activated by putting an initial vector at the input of a layer, the network evolves a two-pattern stable state with each pattern at the output of one layer. Thus, the network involves two layers of interaction between each other

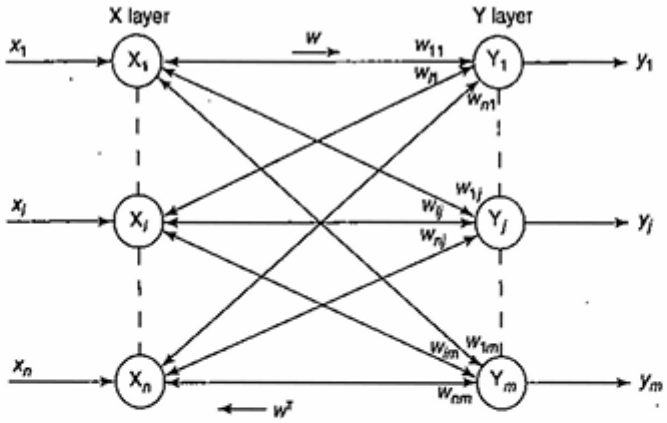


Figure 4-6 Bidirectional associative memory net.

The two bivalent forms of BAM are found to be related with each other, i.e., binary and bipolar. The weights in both the cases are found as the sum of the outer products of the bipolar form of the given training vector pairs. In case of BAM, a definite nonzero threshold is assigned. Thus, the activation function is a step function, with the defined nonzero threshold. When compared, to the binary vectors, bipolar vectors improve the performance of the net to a large extent.

4.5.3.1 Determination of Weights

Let the input vectors be denoted by $s(p)$ and target vectors by $t(p)$, $p = 1, \ldots, P$. Then the weight matrix to store a set of input and target vectors, where

$$s(p) = (s_1(p), \ldots, s_i(p), \ldots, s_n(p))$$
$$t(p) = (t_1(p), \ldots, t_j(p), \ldots, t_m(p))$$

can be determined by Hebb rule training algorithm discussed in Section 4.2.1. In case of input vectors being binary, the weight matrix $W = \{w_{ij}\}$ is given by

$$w_{ij} = \sum_{p=1}^{P} [2s_i(p) - 1][2t_j(p) - 1]$$

On the other hand, when the input vectors are bipolar, the weight matrix $W = \{w_{ij}\}$ can be defined as

$$w_{ij} = \sum_{p=1}^{P} s_i(p) t_j(p)$$

The weights matrix in both the cases is going to be in bipolar form neither the input vectors are in binary or not. The formulas mentioned above can be directly applied to the determination of weights of a BAM.

4.5.3.2 Activation Functions for BAM

The step activation function with a nonzero threshold is used as the activation function for discrete BAM network. The activation function is based on whether the input target vector pairs used are binary or bipolar. The activation function for the Y layer

1. with binary input vectors is

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > 0 \\ y_j & \text{if } y_{inj} = 0 \\ 0 & \text{if } y_{inj} < 0 \end{cases}$$

2. with bipolar input vectors is

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > \theta_j \\ y_j & \text{if } y_{inj} = \theta_j \\ -1 & \text{if } y_{inj} < \theta_j \end{cases}$$

The activation function for the X layer

1. with binary input vectors is

$$x_i = \begin{cases} 1 & \text{if } x_{ini} > 0 \\ x_i & \text{if } x_{ini} = 0 \\ 0 & \text{if } x_{ini} < 0 \end{cases}$$

2. with bipolar input vectors is

$$x_i = \begin{cases} 1 & \text{if } x_{ini} > \theta_i \\ x_i & \text{if } x_{ini} = \theta_i \\ -1 & \text{if } x_{ini} < \theta_i \end{cases}$$

It may be noted that if the threshold value is equal to that of the net input calculated, then the previous output value calculated is left as the activation of that unit. At a particular time instant, signals are sent only from one layer to the other and not in both the directions.

4.5.3.3 Testing Algorithm for Discrete BAM

The testing algorithm is used to test the noisy patterns entering into the network. Based on the training algorithm, weights are determined, by means of which net input is calculated for the given test pattern and activations is applied over it, to recognize the test patterns. The testing algorithm for the net is as follows:

Step 0: Initialize the weights to store p vectors. Also initialize all the activations to zero.

Step 1: Perform Steps 2–6 for each testing input.

Step 2: Set the activations of X layer to current input pattern, i.e., presenting the input pattern x to X layer and similarly presenting the input pattern y to Y layer. Even though, it is bidirectional memory, at one time step, signals can be sent from only one layer. So, either of the input patterns may be the zero vector.

Step 3: Perform Steps 4–6 when the activations are not converged.

Step 4: Update the activations of units in Y layer. Calculate the net input,

$$y_{inj} = \sum_{i=1}^{n} x_i w_{ij}$$

Applying the activations (as in Section 4.5.3.2), we obtain

$$y_j = f(y_{inj})$$

Send this signal to the X layer.

Step 5: Update the activations of units in X layer. Calculate the net input,

$$x_{ini} = \sum_{j=1}^{m} y_j w_{ij}$$

Apply the activations over the net input,

$$x_i = f(x_{ini})$$

Send this signal to the Y layer.

Step 6: Test for convergence of the net. The convergence occurs if the activation vectors x and y reach equilibrium. If this occurs then stop, otherwise, continue.

4.5.4 Continuous BAM

A continuous BAM transforms the input smoothly and continuously in the range 0–1 using logistic sigmoid functions as the activation functions for all units. The logistic sigmoidal function may be either binary sigmoidal function or bipolar sigmoidal function. When a bipolar sigmoidal function with a high gain is chosen, then the continuous BAM might converge to a state of vectors which will approach vertices of the cube. When that state of the vector approaches it acts like a discrete BAM.

If the input vectors are binary, $(s(p), t(p))$, $p = 1$ to P, the weights are determined using the formula

$$w_{ij} = \sum_{p=1}^{P} [2s_i(p) - 1][2t_j(p) - 1]$$

i.e., even though the input vectors are binary, the weight matrix is bipolar. The activation function used here is the logistic sigmoidal function. If it is binary logistic function, then the activation function is

$$f(y_{inj}) = \frac{1}{1 + e^{-y_{inj}}}$$

If the activation function used is a bipolar logistic function, then the function is defined as

$$f(y_{inj}) = \frac{2}{1 + e^{-y_{inj}}} - 1 = \frac{1 - e^{-y_{inj}}}{1 + e^{-y_{inj}}}$$

These activation functions are applied over the net input to calculate the output. The net input can be calculated with a bias included, i.e.,

$$y_{inj} = b_j + \sum_i x_i w_{ij}$$

and all these formulas apply for the units in X layer also.
